NEILVENT VICTOR CHENARIAS (2510101007)
CHAPTER 7

Latihan 1: Lookup cost

python ukur_pencarian.py

```
Lookup per susunan         : 200
Node per jaringan          : 8
Kunci kurs terdaftar       : 26

Susunan client-server
  Pesan median             : 2.0
  Pesan persentil ke-95    : 2.0
  Pesan terbesar           : 2.0
  Hop median               : 1.0
  Hop persentil ke-95      : 1.0
  Hop terbesar             : 1.0
  Lookup terjawab          : 200 dari 200
  Dijawab node sendiri     : 0 dari 200
  Sisi jaringan            : 7

Susunan unstructured
  Pesan median             : 19.0
  Pesan persentil ke-95    : 20.0
  Pesan terbesar           : 20.0
  Hop median               : 2.0
  Hop persentil ke-95      : 3.0
  Hop terbesar             : 3.0
  Lookup terjawab          : 200 dari 200
  Dijawab node sendiri     : 25 dari 200
  Sisi jaringan            : 12

Susunan structured
  Pesan median             : 4.0
  Pesan persentil ke-95    : 6.0
  Pesan terbesar           : 8.0
  Hop median               : 2.0
  Hop persentil ke-95      : 3.0
  Hop terbesar             : 4.0
  Lookup terjawab          : 200 dari 200
  Dijawab node sendiri     : 29 dari 200
  Sisi jaringan            : 13
```

Setelah batas lompatan jadi 2:
Lookup per susunan : 200
Node per jaringan : 8
Kunci kurs terdaftar : 26

Susunan client-server
  Pesan median : 2.0
  Pesan persentil ke-95 : 2.0
  Pesan terbesar : 2.0
  Hop median : 1.0
  Hop persentil ke-95 : 1.0
  Hop terbesar : 1.0
  Lookup terjawab : 200 dari 200
  Dijawab node sendiri : 0 dari 200
  Sisi jaringan : 7

Susunan unstructured
  Pesan median : 9.0
  Pesan persentil ke-95 : 17.0
  Pesan terbesar : 18.0
  Hop median : 1.0
  Hop persentil ke-95 : 2.0
  Hop terbesar : 2.0
  Lookup terjawab : 176 dari 200
  Dijawab node sendiri : 25 dari 200
  Sisi jaringan : 12

Susunan structured
  Pesan median : 4.0
  Pesan persentil ke-95 : 6.0
  Pesan terbesar : 8.0
  Hop median : 2.0
  Hop persentil ke-95 : 3.0
  Hop terbesar : 4.0
  Lookup terjawab : 200 dari 200
  Dijawab node sendiri : 29 dari 200
  Sisi jaringan : 13

python susunan.py cari tetangga EUR/IDR
Susunan : unstructured
Peminta : node 3

Kunci : EUR/IDR
Pemegang kunci : node 5
Nilai ditemukan : 20368.91
Jumlah pesan : 20
Jumlah hop : 3

python susunan.py cari cincin EUR/IDR
Susunan : structured
Peminta : node 3
Kunci : EUR/IDR
Pemegang kunci : node 0
Nilai ditemukan : 20368.91
Jumlah pesan : 6
Jumlah hop : 3

python uji_pencarian_sebagian.py
Potongan nama dicari : USD
Kunci yang memuatnya : 1 dari 26
Client-server ketemu : 1 (2 pesan)
Unstructured ketemu : 1 (14 pesan)
Structured ketemu : 0 (2 pesan)

python ukur_pencarian.py setelah berikut = *self*.jari_terdekat(sekarang, *sasaran*) or penerus
diubah jadi berikut = penerus

Lookup per susunan : 200
Node per jaringan : 8
Kunci kurs terdaftar : 26

Susunan client-server
Pesan median : 2.0
Pesan persentil ke-95 : 2.0
Pesan terbesar : 2.0
Hop median : 1.0
Hop persentil ke-95 : 1.0
Hop terbesar : 1.0
Lookup terjawab : 200 dari 200
Dijawab node sendiri : 0 dari 200
Sisi jaringan : 7

Susunan unstructured
  Pesan median : 19.0
  Pesan persentil ke-95 : 20.0
  Pesan terbesar : 20.0
  Hop median : 2.0
  Hop persentil ke-95 : 3.0
  Hop terbesar : 3.0
  Lookup terjawab : 200 dari 200
  Dijawab node sendiri : 25 dari 200
  Sisi jaringan : 12

Susunan structured
  Pesan median : 8.0
  Pesan persentil ke-95 : 14.0
  Pesan terbesar : 14.0
  Hop median : 4.0
  Hop persentil ke-95 : 7.0
  Hop terbesar : 7.0
  Lookup terjawab : 200 dari 200
  Dijawab node sendiri : 29 dari 200
  Sisi jaringan : 13

Yang dilaporkan	**Client-server**	**Unstructured**	**Structured**
Pesan median	2	19	4
Pesan terbesar	2	20	8
Hop terbesar	1	3	4
Pesan dengan batas dua	2	9	4
Terjawab dengan batas dua	200	176	200
Hope median tanpa finger table	1	2	4

1. Penyebab Sebaran Pesan *Unstructured* Hampir Rata vs *Structured* Melebar (4 sampai 8)

- **Rujukan Angka:** Pada susunan *unstructured*, pesan median berada di angka 19.0 dengan pesan terbesar 20.0 (rentang sangat sempit). Pada susunan *structured*, pesan median adalah 4.0 dan pesan terbesar 8.0 (rentang melebar).
- **Penjelasan:** Susunan *unstructured* menggunakan mekanisme *flooding* (banjir pesan ke seluruh tetangga). Setiap *lookup* langsung menyebarkan pesan ke hampir seluruh sisi

jaringan, sehingga jumlah pesan selalu mendekati batas maksimum total sisi jaringan (12 sisi/20 pesan) tanpa memedulikan lokasi kunci. Sebaliknya, pada susunan *structured* (cincin DHT), pesan dikirimkan secara terarah via *routing*. Jumlah pesan dan *hop* berbanding lurus dengan jarak *node* peminta ke *node* pemegang kunci pada cincin, sehingga nilainya bervariasi bergantung pada letak kunci yang dicari (mulai dari 4 pesan/2 *hop* hingga 8 pesan/4 *hop*).

2.Hal yang Diperoleh dan Hilang Saat Batas *Hop* Diturunkan Menjadi Dua

- **Rujukan Angka (*Unstructured*):**
    - **Yang diperoleh:** Efisiensi lalu lintas jaringan, di mana **pesan median berkurang drastis dari 19.0 menjadi 9.0**.
    - **Yang hilang:** Kepastian pencarian, di mana **jumlah *lookup* terjawab turun dari 200 menjadi 176 dari 200** (sebanyak 24 *lookup* gagal terjangkau/hilang).
- **Penjelasan:** Memangkas batas lompatan (*TTL/Time-to-Live*) membatasi jangkauan *flooding* pesan sehingga menghemat penggunaan bandwidth jaringan, namun mengorbankan kelengkapan pencarian (*search recall*) karena *node* yang jaraknya lebih dari 2 *hop* tidak akan pernah terjangkau.

3.Perbandingan *Hop Structured* dengan/tanpa *Finger Table* dan Alasan Dibutuhkannya Jalan Pintas

- **Rujukan Angka:** *Hop* median susunan *structured* **naik dari 2.0 menjadi 4.0** (dengan *hop* terbesar naik dari **4.0 menjadi 7.0**, serta pesan median naik dari **4.0 menjadi 8.0**) ketika *finger table* dimatikan.
- **Alasan Dibutuhkan Jalan Pintas (*Finger Table*):** Meskipun lokasi kunci/ID penanggung jawab sudah dapat dihitung matematis menggunakan fungsi *hash*, tanpa *finger table node* hanya memiliki koneksi langsung ke penerus terdekatnya (*successor*). Pencarian dipaksa merayap linier satu per satu mengelilingi cincin ($O(N)$). *Finger table* menyediakan "jalan pintas" (*shortcut*) bertingkat eksponensial yang memungkinkan pencarian melompati separuh jaringan sekaligus, sehingga memangkas kompleksitas pencarian secara signifikan menjadi skala logaritmik ($O(\log N)$).

Latihan 2: Failure Resilience

python uji_ketahanan.py

Lookup per percobaan : 200
Node per jaringan : 8
Giliran node yang mati : 0 sampai 7

Susunan client-server

Seluruh node hidup : 200 dari 200

Node 0 mati : 0.0 persen

Rata-rata satu node mati : 87.5 persen

Terburuk satu node mati : 0.0 persen (node 0)

Kunci pada node terburuk : 26

Susunan unstructured

Seluruh node hidup : 200 dari 200

Node 0 mati : 94.0 persen

Rata-rata satu node mati : 83.5 persen

Terburuk satu node mati : 55.2 persen (node 1)

Kunci pada node terburuk : 4

Susunan structured

Seluruh node hidup : 200 dari 200

Node 0 mati : 86.0 persen

Rata-rata satu node mati : 87.9 persen

Terburuk satu node mati : 71.5 persen (node 6)

Kunci pada node terburuk : 7

python tetangga.py

unstructured node 8 sisi 12 kunci 26

node 0 neighbor [1, 4, 7] kunci 1

node 1 neighbor [0, 2, 4, 5, 7] kunci 4

node 2 neighbor [1, 3] kunci 1

node 3 neighbor [2, 7] kunci 1

node 4 neighbor [0, 1, 6, 7] kunci 4

node 5 neighbor [1] kunci 6

node 6 neighbor [4, 7] kunci 5

node 7 neighbor [0, 1, 3, 4, 6] kunci 4

Lookup EUR/IDR dari node 3 : LookupResult(EUR/IDR, nilai=20368.91, pesan=20, hop=3)

python cincin.py

structured node 8 sisi 13 kunci 26

node 0 pengenal 5425 jari [6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 5] kunci 4

node 6 pengenal 27881 jari [3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 5, 4, 0] kunci 7

node 3 pengenal 33413 jari [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 4, 0] kunci 3

node 5 pengenal 42962 jari [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 4, 4, 4, 0, 6] kunci 7

node 1 pengenal 44208 jari [4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 7, 0, 6] kunci 2

node 4 pengenal 51768 jari [7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 2, 0, 0, 6] kunci 2

node 7 pengenal 54466 jari [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 6] kunci 0

node 2 pengenal 57505 jari [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 6, 6] kunci 1

Pengenal kunci EUR/IDR : 63784

Lookup EUR/IDR dari node 3 : LookupResult(EUR/IDR, nilai=20368.91, pesan=6, hop=3)

Yang dilaporkan	Client-server	Unstructured	Structured
Node 0 mati	0 persen	94 persen	86 persen
Rata-rata satu node mati	87,5 persen	83,5 persen	87,9 persen
Terburuk satu node mati	0 persen	55,2 persen	71,5 persen
Nomor node terburuk	0	1	6

1. Mengapa Rata-Rata *Client-Server* Menyesatkan dan Baris yang Seharusnya Dibaca

- **Rujukan Angka:** Rata-rata *client-server* bernilai tinggi (**87,5 persen**), namun ketika *node* 0 mati, tingkat keberhasilan *lookup* anjlok hingga **0,0 persen**.
- **Penjelasan:** Angka rata-rata **87,5%** tersebut menyesatkan karena didapat dari kondisi ideal saat 7 *client node* lainnya yang mati (masing-masing 100% sukses, sehingga (7 x 100% + 1 x 0%) / 8 = 87,5%). Nilai rata-rata ini menyembunyikan titik kegagalan tunggal (*single point of failure*). Jika *node* 0 (server pusat) mati, seluruh sistem kolaps total. Baris yang seharusnya dibaca untuk mengukur ketahanan sejati adalah **Terburuk satu *node* mati** (0,0 persen).

2. Mengapa Matinya *Node* 1 Sangat Merugikan pada Susunan *Unstructured*

- **Rujukan Angka:** *Node* 1 hanya memegang **4 kunci**, tetapi saat mati, persentase *lookup* terjawab turun paling drastis menjadi **55,2 persen** (kehilangan hampir separuh akses pencarian).
- **Penjelasan:** Berdasarkan luaran python tetangga.py, *node* 1 terhubung ke 5 tetangga [0, 2, 4, 5, 7] dan menjadi **satu-satunya jembatan penghubung (*single bridge/articulation point*) bagi *node* 5**, yang mana *node* 5 hanya memiliki 1 *neighbor* [1] dan menampung 6 kunci. Matinya *node* 1 tidak hanya menghilangkan 4 kunci di *node* 1, tetapi juga memutuskan koneksi ke *node* 5 beserta 6 kuncinya, sekaligus memutus jalur *flooding* pencarian antar-kluster topologi.

3. Perubahan untuk Menaikkan Angka Terburuk *Structured* dan Ongkosnya

- **Perubahan:** Menerapkan mekanisme **Replikasi Kunci (*Key Replication*)**, yaitu menyimpan salinan setiap kunci dan nilainya pada beberapa *node* penerus terdekat (*successor set*, misal *k node* setelahnya pada cincin).
- **Ongkos yang Harus Dibayar:**
    1. **Konsumsi Memori & Penyimpanan:** Setiap *node* harus menampung data ganda/duplikat dari *node* lain.

2. **Overhead Lalu Lintas Jaringan:** Terjadi penambahan pesan sinkronisasi (*write overhead*) untuk menjaga konsistensi data saat ada pembaruan atau penambahan kunci baru pada sistem.

Latihan 3: Message Scalability

python ukur_penskalaan.py

Lookup per ukuran : 200

Ukuran jaringan diuji : 8 lalu 16 node

Pertumbuhan jumlah node : 2.00 kali

Pertumbuhan logaritmanya : 1.33 kali

Susunan client-server

Pesan median 8 node : 2.0

Pesan median 16 node : 2.0

Pertumbuhan pesan : 1.00 kali

Hop median 8 node : 1.0

Hop median 16 node : 1.0

Sisi jaringan : 7 lalu 15

Lookup terjawab : 200 lalu 200 dari 200

Susunan unstructured

Pesan median 8 node : 19.0

Pesan median 16 node : 35.0

Pertumbuhan pesan : 1.84 kali

  Hop median 8 node   : 2.0

  Hop median 16 node   : 2.0

  Sisi jaringan   : 12 lalu 24

  Lookup terjawab   : 200 lalu 199 dari 200

Susunan structured

  Pesan median 8 node   : 4.0

  Pesan median 16 node   : 6.0

  Pertumbuhan pesan   : 1.50 kali

  Hop median 8 node   : 2.0

  Hop median 16 node   : 3.0

  Sisi jaringan   : 13 lalu 35

  Lookup terjawab   : 200 lalu 200 dari 200

python ukur_pencarian.py 32

Lookup per susunan   : 200

Node per jaringan   : 32

Kunci kurs terdaftar   : 26

Susunan client-server

  Pesan median   : 2.0

  Pesan persentil ke-95   : 2.0

  Pesan terbesar   : 2.0

Hop median : 1.0

Hop persentil ke-95 : 1.0

Hop terbesar : 1.0

Lookup terjawab : 200 dari 200

Dijawab node sendiri : 0 dari 200

Sisi jaringan : 31

Susunan unstructured

Pesan median : 52.0

Pesan persentil ke-95 : 65.0

Pesan terbesar : 67.0

Hop median : 3.0

Hop persentil ke-95 : 4.0

Hop terbesar : 4.0

Lookup terjawab : 172 dari 200

Dijawab node sendiri : 6 dari 200

Sisi jaringan : 48

Susunan structured

Pesan median : 6.0

Pesan persentil ke-95 : 10.0

Pesan terbesar : 10.0

Hop median : 3.0

Hop persentil ke-95 : 5.0

Hop terbesar : 5.0

Lookup terjawab : 200 dari 200

Dijawab node sendiri : 11 dari 200

Sisi jaringan : 86

Yang dilaporkan	**Client-server**	**Unstructured**	**Structured**
Pesan median 8 node	2	19	4
Pesan median 16 node	2	35	6
Pesan median 32 node	2	52	6
Pertumbuhan pesan	100 kali	1.84 kali	1.50 kali
Terjawab pada 16 node	200	199	200

1.Susunan yang Pertumbuhannya Paling Dekat dengan 2.00 Kali dan Sebabnya

- **Rujukan Angka:** Pertumbuhan pesan susunan ***Unstructured*** tercatat sebesar **1.84 kali**, paling mendekati angka pembanding **2.00 kali** (pertumbuhan linier sebanding jumlah *node*).
- **Penjelasan:** Susunan *unstructured* bekerja menggunakan metode *flooding* (banjir pesan ke seluruh tetangga terhubung). Saat jumlah *node* dan sisi jaringan bertambah dua kali lipat (dari 8 *node*/12 sisi menjadi 16 *node*/24 sisi), setiap pencarian memaksa pesan menyebar ke hampir seluruh sisi jaringan baru tersebut. Akibatnya, jumlah pesan membengkak secara linier berbanding lurus dengan skala jaringan.

2.Penyebab Pertumbuhan *Structured* Tercatat 1.50 Kali (Bukan 1.33 Kali)

- **Rujukan Angka:** Pertumbuhan pesan *Structured* tercatat **1.50 kali** (naik dari 4.0 menjadi 6.0 pesan), sedangkan estimasi logaritmik idealnya adalah **1.33 kali** ($\log_2(16)/\log_2(8) = 4/3 \approx 1.33$).
- **Penjelasan:** Discrepancy ini terjadi karena skala ukuran jaringan yang diuji masih sangat kecil (hanya dari 8 ke 16 *node*). Pada skala kecil, *overhead* penyiapan *routing* cincin dan pembulatan hop diskrit pada *finger table* memberikan dampak proporsional yang lebih besar dibanding skala teoritisnya. Skalabilitas logaritmik sejati ($O(\log N)$) baru akan

mendekati rasio teoritis secara presisi saat jaringan bertambah besar ke skala ratusan atau ribuan *node*.

3.Keputusan Susunan untuk Sejuta *Node* ($1.000.000$**) dan 3 Angka Dasarnya**

- **Pilihan Susunan: Susunan *Structured* (DHT/Chord-based)**.
- **3 Angka Dasar Keputusan:**
    1. **Skalabilitas Pesan Logaritmik (Latihan 3):** Jumlah pesan *Structured* hanya tumbuh **1.50 kali** saat jaringan membesar dua kali lipat, dan pesan mediannya pada 32 *node* tetap sangat efisien di angka **6.0 pesan** (berbanding jauh dengan *Unstructured* yang membengkak ke **52.0 pesan**).
    2. **Keandalan Pencarian / *Recall* 100% (Latihan 3):** Jumlah *lookup* terjawab pada *Structured* selalu konsisten sempurna di angka **200 dari 200** (100%), sedangkan *Unstructured* drop ke **172 dari 200** karena terbatasi *TTL/hop limit*.
    3. **Ketahanan Kegagalan / *Resilience* (Latihan 2):** Saat terjadi kegagalan satu *node*, skenario terburuk *Structured* masih mampu menjawab **71.5% *lookup*** (berbanding dengan *Client-server* yang anjlok ke **0.0%** alias *total breakdown* saat *server* pusat mati).